Advanced Artificial Intelligence: Algorithms & Implementations

Comprehensive Study Guide

April 28, 2026

Abstract

This document serves as a comprehensive study guide and reference manual for Advanced Artificial Intelligence. It covers fundamental search strategies, constraint satisfaction problems, probabilistic reasoning, adversarial games, and genetic algorithms. The guide includes detailed theoretical explanations, pseudocode, and pedagogical Python implementations designed to clarify complex algorithmic concepts such as A* Search, Minimax with Alpha-Beta Pruning, Bellman-Ford, and Bayesian Networks.

Contents

1 Constraint Satisfaction Problems (CSP) & Backtracking **4**
1.1 The N-Queens Problem . . . 4
1.1.1 Algorithm Logic: Backtracking . . . 4
1.1.2 Python Implementation . . . 4
1.1.3 C++ Implementation . . . 5

2 Heuristic Search Algorithms **6**
2.1 Best-First Search (BFS) . . . 6
2.1.1 Best-First Search Code (Simplest Iterative Method) . . . 7
2.1.2 Best-First Search Code (Using Priority Queue) . . . 7
2.2 0/1 Knapsack Problem using Heuristic Search . . . 8

3 Advanced Heuristic Search & Algorithm Debugging **9**
3.1 Priority Queue Mechanisms in Best-First Search . . . 9
3.2 A* Algorithm for the Traveling Salesperson Problem (TSP) . . . 10
3.2.1 A* Python Implementation with Complex Heuristic . . . 10

4 Local Search Algorithms **12**
4.1 Hill Climbing for N-Queens . . . 12
4.2 Hill Climbing & Iterative Hill Climbing for 8-Puzzle . . . 13

5 Advanced Algorithms: 2D Matrices and Genetic Algorithms **14**
5.1 2D Matrix Implementation for Local Search . . . 14
5.2 Genetic Algorithm (GA) for Travelling Salesman Problem (TSP) . . . 15

6 Constraint Satisfaction Problems (CSP) **17**
6.1 Crossword Puzzle as a CSP . . . 17
6.2 Solving CSP using Backtracking . . . 17
6.3 Arc Consistency (AC-3) Algorithm . . . 17
6.4 Sudoku Solving using CSP Heuristics . . . 19

7 Graph Algorithms: Shortest Paths **21**
7.1 Dijkstra's Algorithm . . . 21
7.2 Bellman-Ford Algorithm . . . 22
7.3 Floyd-Warshall Algorithm (All-Pairs Shortest Path) . . . 22

8 Optimization & Search Problems **23**
8.1 Brute-Force TSP using Permutations . . . 23
8.2 Fractional vs. 0/1 Knapsack . . . 23
8.3 Algorithm Comparison Summary . . . 24

9 Probabilistic Reasoning & Bayesian Networks **24**
9.1 Bayesian Networks Overview . . . 24
9.2 Constructing Bayesian Networks in Python . . . 24
9.3 Pure Python Bayesian Network Implementation . . . 26

10 Uncertainty & Fuzzy Logic **27**
10.1 Logical Deduction vs Induction . . . 27
10.2 Why Probability over First-Order Logic (FOL)? . . . 27
10.3 Bayesian Inferences . . . 27
10.4 Fuzzy Logic . . . 27

11 Advanced CSP: MRV and LCV **28**
11.1 MRV (Minimum Remaining Values) . . . 28
11.2 LCV (Least Constraining Value) . . . 28

12 Local Search Algorithms **28**
12.1 Hill Climbing Algorithm (Steepest Ascent) . . . 28
12.2 Iterated (Random-Restart) Hill Climbing . . . 29

13 Crossword Puzzle as a CSP **30**
13.1 Backtracking (Smart Hit & Trial) . . . 30
13.2 AC-3 (Arc Consistency 3) . . . 30

14 Debugging Notes: 8-Puzzle Recursive Implementation **30**

15 Adversarial Search & Game Playing **31**
15.1 Minimax Algorithm . . . 31
15.2 Alpha-Beta Pruning . . . 31

16 The 8-Queens Problem **32**
16.1 Smart Board Representation . . . 32
16.2 The Cost Function (Referee) . . . 32
16.3 Algorithm Logic . . . 33

17 Graph Algorithms & Optimization **34**
17.1 A* Algorithm (Pathfinding) . . . 34
17.2 Bellman-Ford Algorithm . . . 34

18 Genetic Algorithms (Traveling Salesperson Problem) **34**

19 Advanced CSP Formulation: Timetable Scheduling **34**
19.1 AC-3 (Arc Consistency 3) . . . 35

20 Debugging Notes: 8-Puzzle Recursive Implementation **35**

21 Adversarial Search & Game Playing **35**
21.1 Minimax Algorithm 35
21.2 Alpha-Beta Pruning 36

22 The 8-Queens Problem **37**
22.1 Smart Board Representation 37
22.2 The Cost Function (Referee) 37
22.3 Algorithm Logic 37

23 Graph Algorithms & Optimization **38**
23.1 A* Algorithm (Pathfinding) 38
23.2 Bellman-Ford Algorithm 38

24 Genetic Algorithms (Traveling Salesperson Problem) **39**

25 Advanced CSP Formulation: Timetable Scheduling **39**

1 Constraint Satisfaction Problems (CSP) & Backtracking

1.1 The N-Queens Problem

[N-Queens] The goal is to place N queens on an $N \times N$ chessboard so that no two queens attack each other. This means no two queens can share the same row, column, or diagonal.

In Artificial Intelligence, this is treated as a state-space search problem, typically solved using backtracking.

1.1.1 Algorithm Logic: Backtracking

1. Start with an empty board.
2. Place a queen in the first valid column of the current row.
3. **Check Constraints:** Is the queen safe from queens in previous rows?
4. **Recurse:** Move to the next row and repeat.
5. **Backtrack:** If you reach a row where no column is safe, you have hit a "dead end." Go back to the previous row, remove that queen, and try the next column.

1.1.2 Python Implementation

Python is preferred in AI for its readability. This uses a simple recursive function.

```
def print_board(board, n):
    for i in range(n):
        for j in range(n):
            print("Q" if board[i][j] == 1 else ".", end=" ")
        print()
    print("-" * (n * 2))

def is_safe(board, row, col, n):
    # 1. Check this column on upper side
    for i in range(row):
        if board[i][col] == 1:
            return False

    # 2. Check upper-left diagonal
    i, j = row, col
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1

    # 3. Check upper-right diagonal
    i, j = row, col
    while i >= 0 and j < n:
        if board[i][j] == 1:
            return False
        i -= 1
        j += 1

    return True

def solve_n_queens_util(board, row, n):
    if row >= n:
```

```python
        return True

    for col in range(n):
        if is_safe(board, row, col, n):
            board[row][col] = 1 # Place the queen

            if solve_n_queens_util(board, row + 1, n):
                return True

            board[row][col] = 0 # BACKTRACK

    return False

def solve_n_queens(n):
    board = [[0 for _ in range(n)] for _ in range(n)]
    if not solve_n_queens_util(board, 0, n):
        print("Solution does not exist")
        return
    print_board(board, n)

if __name__ == "__main__":
    solve_n_queens(4)
```

Listing 1: N-Queens Python Code

1.1.3 C++ Implementation

C++ is used when performance is critical. The logic is identical, but memory management is explicit.

```cpp
#include <iostream>
#include <vector>

using namespace std;

void printBoard(const vector<vector<int>>& board, int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (board[i][j]) cout << "Q ";
            else cout << ". ";
        }
        cout << endl;
    }
}

bool isSafe(const vector<vector<int>>& board, int row, int col, int n)
   {
    int i, j;
    for (i = 0; i < row; i++)
        if (board[i][col]) return false;

    for (i = row, j = col; i >= 0 && j >= 0; i--, j--)
        if (board[i][j]) return false;

    for (i = row, j = col; i >= 0 && j < n; i--, j++)
        if (board[i][j]) return false;

    return true;
```

```
}

bool solveNQueensUtil(vector<vector<int>>& board, int row, int n) {
    if (row >= n) return true;

    for (int col = 0; col < n; col++) {
        if (isSafe(board, row, col, n)) {
            board[row][col] = 1;
            if (solveNQueensUtil(board, row + 1, n)) return true;
            board[row][col] = 0; // BACKTRACK
        }
    }
    return false;
}

void solveNQueens(int n) {
    vector<vector<int>> board(n, vector<int>(n, 0));
    if (!solveNQueensUtil(board, 0, n)) {
        cout << "Solution does not exist";
        return;
    }
    printBoard(board, n);
}

int main() {
    int n = 4;
    solveNQueens(n);
    return 0;
}
```

Listing 2: N-Queens C++ Code

AI Course Importance

State Space Search: The board represents a "state." The algorithm explores the "space" of possible states to find a goal state.
Heuristics: Advanced AI uses heuristics (e.g., Min-Conflicts) instead of blind backtracking to guess optimal columns, significantly speeding up large boards. (Bitmasking can further optimize the code.)

2 Heuristic Search Algorithms

2.1 Best-First Search (BFS)

[Best-First Search] Given an undirected graph, Best-First Search (Greedy BFS) finds the path to the goal by always selecting the node that appears closest to the goal, dictated by the lowest heuristic value (h), ignoring the cost of the path so far.

Consider Nodes A to G with Heuristics to G: A=8, B=6, C=5, D=3, E=2, F=1, G=0.

1. **Start:** A ($h = 8$). Neighbors: B(6), C(5). Choose C ($5 < 6$).
2. **Current:** C ($h = 5$). Neighbors: E(2), F(1). Choose F ($1 < 2$).
3. **Current:** F ($h = 1$). Neighbor: G(0). Choose G.
4. **Path:** $A \to C \to F \to G$.

2.1.1 Best-First Search Code (Simplest Iterative Method)

The algorithm uses the heuristic purely for decision making (choosing the next node), while calculating edge weights only to compute total path cost.

```
heuristics = {'A': 8, 'B': 6, 'C': 5, 'D': 3, 'E': 2, 'F': 1, 'G': 0}

graph = {
    'A': [('B', 4), ('C', 3)],
    'B': [('D', 5)],
    'C': [('E', 6), ('F', 2)],
    'D': [],
    'E': [('G', 4)],
    'F': [('G', 1)],
    'G': []
}

def solve_best_first_search():
    current_node = 'A'
    goal_node = 'G'
    path = ['A']
    total_cost = 0

    while current_node != goal_node:
        neighbors = graph[current_node]
        if not neighbors:
            print("Dead end reached!")
            return

        best_neighbor = None
        best_neighbor_cost = 0
        min_h = 9999

        for neighbor, cost in neighbors:
            h_val = heuristics[neighbor]
            if h_val < min_h:
                min_h = h_val
                best_neighbor = neighbor
                best_neighbor_cost = cost

        current_node = best_neighbor
        total_cost += best_neighbor_cost
        path.append(current_node)

    print(f"Final Path: {path}")
    print(f"Total Cost: {total_cost}")

solve_best_first_search()
```

Listing 3: Simplest Best First Search Python

2.1.2 Best-First Search Code (Using Priority Queue)

For a robust implementation, a Priority Queue (like Python's `heapq` or C++'s `priority_queue`) should sort nodes solely by heuristic.

```
import heapq

def solve():
```

```
    heuristics = {'A': 8, 'B': 6, 'C': 5, 'D': 3, 'E': 2, 'F': 1, 'G':
        0}
    graph = {
        'A': [('B', 4), ('C', 3)], 'B': [('D', 5)], 'C': [('E', 6), ('F
            ', 2)],
        'D': [], 'E': [('G', 4)], 'F': [('G', 1)], 'G': []
    }

    queue = []
    heapq.heappush(queue, (heuristics['A'], 'A', ['A'], 0))
    visited = set()

    while queue:
        h, current, path, cost = heapq.heappop(queue)
        if current in visited: continue
        visited.add(current)

        if current == 'G':
            print("Final Path:", path)
            print("Total Cost:", cost)
            return

        for neighbor, edge_weight in graph[current]:
            if neighbor not in visited:
                new_path = path + [neighbor]
                new_cost = cost + edge_weight
                heapq.heappush(queue, (heuristics[neighbor], neighbor,
                    new_path, new_cost))

solve()
```

Listing 4: Best First Search using heapq

2.2 0/1 Knapsack Problem using Heuristic Search

The 0/1 Knapsack Problem seeks to maximize value without exceeding weight capacity.

In AI, a heuristic search using Value-to-Weight ratio (v_i/w_i) forms a Greedy Search. It makes locally optimal choices, which serves as a rapid approximation algorithm for 0/1 Knapsack, though it only guarantees optimality in Fractional Knapsack.

$$h(i) = \frac{\text{Value}_i}{\text{Weight}_i} \tag{1}$$

Algorithm Details:

1. Calculate ratio v/w for every item.
2. Sort all items in descending order of their ratios.
3. Iterate through the sorted list, choosing the item if it fits capacity, skipping it otherwise.

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <iomanip>

using namespace std;
```

```cpp
struct Item {
    string id; int weight; int value; double ratio;
    Item(string id, int w, int v) : id(id), weight(w), value(v) {
        ratio = (double)v / w;
    }
};

bool compareRatio(Item a, Item b) {
    return a.ratio > b.ratio;
}

void solveKnapsackHeuristic(int capacity, vector<Item>& items) {
    sort(items.begin(), items.end(), compareRatio);
    int currentWeight = 0;
    int totalValue = 0;

    for (const auto& item : items) {
        if (currentWeight + item.weight <= capacity) {
            currentWeight += item.weight;
            totalValue += item.value;
            cout << item.id << " PICKED\n";
        } else {
            cout << item.id << " SKIPPED\n";
        }
    }
    cout << "Total Value: " << totalValue << endl;
}

int main() {
    int capacity = 50;
    vector<Item> items = { {"Item A", 10, 60}, {"Item B", 20, 100}, {"
        Item C", 30, 120} };
    solveKnapsackHeuristic(capacity, items);
    return 0;
}
```

Listing 5: 0/1 Knapsack Greedy Heuristic (C++)

AI Perspective on Knapsack Heuristics

While Dynamic Programming (DP) finds the exact solution, its time complexity is $\mathcal{O}(nW)$, making it computationally expensive. The Greedy Heuristic Search operates at $\mathcal{O}(n \log n)$. In domains like AI, game development, or cloud resource allocation, Greedy Search is preferred due to its extreme speed, even if marginally suboptimal.

3 Advanced Heuristic Search & Algorithm Debugging

3.1 Priority Queue Mechanisms in Best-First Search

When implementing Best-First Search using a Priority Queue (like Python's `heapq`), the sorting key must strictly be the heuristic value, not the accumulated path cost.

Common Bug: Using Path Cost in BFS

Mistake: `heapq.heappush(queue, (cost + heuristic[neighbor], ...))`
Correction: `heapq.heappush(queue, (heuristic[neighbor], ...))`
Explanation: Best-First Search makes decisions solely based on the heuristic (h). If you include the path cost (g), you are effectively implementing the A* Algorithm, which behaves differently. The cost should be tracked as a separate variable for final output bookkeeping, but excluded from the sorting tuple's first index.

Common Bug: Visited Set Tracking

Mistake: `visited.add(heuristic_value)`
Correction: `visited.add(node_name)`
Explanation: The `visited` set must track the actual identities of the nodes (e.g., 'A', 'B'), not their heuristic values (e.g., 8, 6). Tracking integers leads to type mismatches and infinite loops during graph traversal.

3.2 A* Algorithm for the Traveling Salesperson Problem (TSP)

[A* Search] A* Search combines the actual accumulated cost from the start node, $g(n)$, with the estimated heuristic cost to the goal, $h(n)$, to minimize the total estimated path cost: $f(n) = g(n) + h(n)$.

For a TSP-like problem where the goal is to visit all cities and return to the start, a complex heuristic $h(X)$ can be defined as the sum of three components:

1. Distance to the nearest unvisited city from the current city.
2. Estimated Minimum Spanning Tree (MST) cost for visiting all remaining unvisited cities.
3. Distance from the nearest unvisited city back to the starting city.

3.2.1 A* Python Implementation with Complex Heuristic

```
import heapq

Graph = {
    "A": [("B", 12), ("D", 19), ("E", 8), ("C", 10)],
    "B": [("A", 12), ("C", 3), ("E", 6), ("D", 7)],
    "C": [("A", 10), ("B", 3), ("D", 2), ("E", 20)],
    "D": [("B", 7), ("A", 19), ("C", 2), ("E", 4)],
    "E": [("D", 4), ("B", 6), ("A", 8), ("C", 20)]
}

def distance_to_nearest_unvisited(current_node, visited):
    nearest_dist = 1e9
    found = False
    for neighbor, weight in Graph[current_node]:
        if neighbor not in visited:
            if weight < nearest_dist:
                nearest_dist = weight
                found = True
    return nearest_dist if found else 0

def MST_cost(visited):
    unvisited_nodes = [n for n in Graph.keys() if n not in visited]
```

```
    if len(unvisited_nodes) < 2: return 0

    mst_cost = 0
    nodes_in_mst = {unvisited_nodes[0]}

    while len(nodes_in_mst) < len(unvisited_nodes):
        min_edge = 1e9
        next_node = None
        for u in nodes_in_mst:
            for v, weight in Graph[u]:
                if v in unvisited_nodes and v not in nodes_in_mst:
                    if weight < min_edge:
                        min_edge = weight
                        next_node = v
        if next_node:
            mst_cost += min_edge
            nodes_in_mst.add(next_node)
        else: break
    return mst_cost

def distance_from_unvisited_to_start(visited, start_node):
    nearest_dist = 1e9
    found = False
    for node in Graph.keys():
        if node not in visited:
            for neighbor, weight in Graph[node]:
                if neighbor == start_node:
                    if weight < nearest_dist:
                        nearest_dist = weight
                        found = True
    return nearest_dist if found else 0

def solve_tsp_astar():
    start_node = "A"
    queue = []

    # h = dist_to_unvisited + MST_cost + dist_to_start
    initial_h = distance_to_nearest_unvisited(start_node, {start_node})
        + MST_cost({start_node}) + distance_from_unvisited_to_start({
       start_node}, start_node)

    heapq.heappush(queue, (initial_h, start_node, 0, [start_node], {
       start_node}))

    while queue:
        f, current_node, g, path, visited = heapq.heappop(queue)

        # Goal check: all nodes visited, return to start
        if len(visited) == len(Graph):
            for neighbor, weight in Graph[current_node]:
                if neighbor == start_node:
                    total_cost = g + weight
                    print("Final Path:", path + [start_node])
                    print("Total Cost:", total_cost)
                    return

        # Generate Neighbors
        for neighbor, weight in Graph[current_node]:
```

```
            if neighbor not in visited:
                new_g = g + weight
                new_path = path + [neighbor]
                new_visited = visited.copy()
                new_visited.add(neighbor)

                h1 = distance_to_nearest_unvisited(neighbor,
                    new_visited)
                h2 = MST_cost(new_visited)
                h3 = distance_from_unvisited_to_start(new_visited,
                    start_node)
                h = h1 + h2 + h3
                new_f = new_g + h

                heapq.heappush(queue, (new_f, neighbor, new_g, new_path
                    , new_visited))

if __name__ == "__main__":
    solve_tsp_astar()
```

Listing 6: A* Algorithm for TSP using MST Heuristic

4 Local Search Algorithms

4.1 Hill Climbing for N-Queens

Hill Climbing is a local search algorithm that continually moves towards the direction of increasing value (or decreasing cost). It is highly prone to getting stuck in Local Maxima/Minima.

N-Queens Representation: Instead of a 2D matrix, we use a 1D array where `Index = Column` and `Value = Row`. Example: `[1, 3, 0, 2]` means Queen 0 is at Row 1, Queen 1 is at Row 3, etc. This automatically enforces the "one queen per column" constraint.

```
import random

def count_attacks(board):
    n = len(board)
    attacks = 0
    for i in range(n):
        for j in range(i + 1, n):
            # Check row collision or diagonal collision
            if board[i] == board[j] or abs(board[i] - board[j]) == abs(
                i - j):
                attacks += 1
    return attacks

def get_best_neighbor(board):
    n = len(board)
    best_board = list(board)
    min_attacks = count_attacks(board)

    for col in range(n):
        original_row = board[col]
        for row in range(n):
            if row != original_row:
                board[col] = row
                current_attacks = count_attacks(board)
                if current_attacks < min_attacks:
```

```
                    min_attacks = current_attacks
                    best_board = list(board)
                board[col] = original_row # Backtrack
    return best_board, min_attacks

def solve_n_queens_hc(n):
    while True: # Random Restart Loop
        current_board = [random.randint(0, n - 1) for _ in range(n)]
        current_attacks = count_attacks(current_board)

        while True: # Climbing Loop
            if current_attacks == 0: return current_board

            neighbor_board, neighbor_attacks = get_best_neighbor(
                current_board)

            if neighbor_attacks >= current_attacks:
                break # Stuck in local maxima, trigger restart

            current_board = neighbor_board
            current_attacks = neighbor_attacks

solution = solve_n_queens_hc(8)
print("Solution Board:", solution)
```

Listing 7: N-Queens Hill Climbing

4.2 Hill Climbing & Iterative Hill Climbing for 8-Puzzle

The 8-Puzzle consists of a 3×3 grid with numbers 1-8 and an empty tile (0). The heuristic $H(X)$ is the number of misplaced tiles compared to the goal state.

[Iterative/Random Restart Hill Climbing] Standard Hill Climbing will fail if it reaches a state where all neighbors have equal or worse heuristic values (Local Maxima). Iterative Hill Climbing mitigates this by allowing a fixed number of retries, shuffling the board to a new random state each time it gets stuck.

```
import random

GOAL = [1, 2, 3, 4, 5, 6, 7, 8, 0]

def get_h(board):
    return sum(1 for i in range(9) if board[i] != 0 and board[i] !=
        GOAL[i])

def get_neighbors(board):
    neighbors = []
    idx = board.index(0)
    row, col = idx // 3, idx % 3
    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    for r, c in moves:
        new_r, new_c = row + r, col + c
        if 0 <= new_r < 3 and 0 <= new_c < 3:
            new_idx = new_r * 3 + new_c
            new_board = list(board)
            new_board[idx], new_board[new_idx] = new_board[new_idx],
                new_board[idx]
            neighbors.append(new_board)
```

```
    return neighbors

def hill_climb(start_node):
    curr = start_node
    while True:
        curr_h = get_h(curr)
        if curr_h == 0: return curr, 0 # Solved

        neighbors = get_neighbors(curr)
        best_neighbor = min(neighbors, key=get_h)
        best_h = get_h(best_neighbor)

        if best_h >= curr_h: return curr, curr_h # Stuck
        curr = best_neighbor

def solve_iterative():
    for i in range(1, 16):
        rand_start = list(GOAL)
        random.shuffle(rand_start)
        final_state, final_h = hill_climb(rand_start)
        if final_h == 0:
            print(f"SOLVED in Iteration {i}!")
            return
    print("Failed to solve in 15 iterations.")

solve_iterative()
```

Listing 8: Iterative Hill Climbing for 8-Puzzle

5 Advanced Algorithms: 2D Matrices and Genetic Algorithms

5.1 2D Matrix Implementation for Local Search

While 1D arrays simplify indexing, a 2D matrix provides a more intuitive representation of grid-based problems like the 8-puzzle. Deep copying becomes essential to avoid mutating the original board state during neighbor generation.

```
import copy
import random

GOAL = [[1, 2, 3], [4, 5, 6], [7, 8, 0]]
START = [[3, 4, 8], [6, 0, 7], [5, 2, 1]]

def get_heuristic(board):
    err = 0
    for i in range(3):
        for j in range(3):
            if board[i][j] != 0 and board[i][j] != GOAL[i][j]:
                err += 1
    return err

def get_zero_location(board):
    for i in range(3):
        for j in range(3):
            if board[i][j] == 0: return i, j
    return -1, -1
```

```python
def move_gen(cur_row, cur_col, board):
    neighbours = []
    moves = [[-1, 0], [1, 0], [0, -1], [0, 1]]
    for move in moves:
        new_r = cur_row + move[0]
        new_c = cur_col + move[1]

        if 0 <= new_r < 3 and 0 <= new_c < 3:
            new_board = copy.deepcopy(board)
            val_at_new = new_board[new_r][new_c]
            new_board[cur_row][cur_col] = val_at_new
            new_board[new_r][new_c] = 0
            neighbours.append(new_board)
    return neighbours

def get_random_board():
    nums = [1, 2, 3, 4, 5, 6, 7, 8, 0]
    random.shuffle(nums)
    return [nums[0:3], nums[3:6], nums[6:9]]

def iterative_hill_climb():
    for i in range(1, 16):
        cur_best_board = get_random_board()
        while True:
            cur_best_val = get_heuristic(cur_best_board)
            if cur_best_val == 0:
                print(f"SOLVED in iteration {i}")
                return

            r, c = get_zero_location(cur_best_board)
            neighbours = move_gen(r, c, cur_best_board)

            best_neighbour_board = neighbours[0]
            best_neighbour_val = get_heuristic(neighbours[0])

            for n in neighbours:
                h = get_heuristic(n)
                if h < best_neighbour_val:
                    best_neighbour_val = h
                    best_neighbour_board = n

            if best_neighbour_val >= cur_best_val:
                print("Stuck at Local Maxima. Restarting...")
                break # Break to trigger next iteration

            cur_best_board = best_neighbour_board
```

Listing 9: Iterative Hill Climbing using 2D Matrix

5.2 Genetic Algorithm (GA) for Travelling Salesman Problem (TSP)

Genetic Algorithms mimic natural selection to solve optimization problems. Key steps include Population Generation, Fitness Calculation, Selection, Crossover, and Mutation.

[Genetic Algorithm Workflow]

1. **Initial Population:** Generate random potential solutions (routes).
2. **Fitness Evaluation:** Calculate the total cost of the route. Lower cost is better.

3. **Selection:** Select the top-performing individuals to become parents.
4. **Crossover:** Combine elements of two parents to form a child.
5. **Mutation:** Introduce random changes (e.g., swapping two cities) to maintain genetic diversity.

```
import random

graph = {
    'A': {'B': 12, 'C': 10, 'D': 19, 'E': 8},
    'B': {'A': 12, 'C': 3,  'D': 7,  'E': 6},
    'C': {'A': 10, 'B': 3,  'D': 2,  'E': 20},
    'D': {'A': 19, 'B': 7,  'C': 2,  'E': 4},
    'E': {'A': 8,  'B': 6,  'C': 20, 'D': 4}
}
cities = ['B', 'C', 'D', 'E']

def get_route_cost(route):
    cost = graph['A'][route[0]]
    for i in range(len(route) - 1):
        cost += graph[route[i]][route[i + 1]]
    cost += graph[route[-1]]['A']
    return cost

def create_population(size=4):
    population = []
    for _ in range(size):
        city_list = list(cities)
        random.shuffle(city_list)
        population.append(city_list)
    return population

def crossover(parent1, parent2):
    child = [parent1[0], parent1[1]]
    for city in parent2:
        if city not in child:
            child.append(city)
    return child

def mutate(route):
    mutate_route = list(route)
    idx1, idx2 = random.sample(range(4), 2)
    mutate_route[idx1], mutate_route[idx2] = mutate_route[idx2],
        mutate_route[idx1]
    return mutate_route

def genetic_algo():
    population = create_population(4)

    for i in range(30):
        population.sort(key=get_route_cost)
        best_par1 = population[0]
        best_par2 = population[1]

        new_child = crossover(best_par1, best_par2)
        child_after_mutation = mutate(new_child) # Mutate child instead
            of parent
```

```
        # Next generation: keep 2 parents, add 2 children
        population = [best_par1, best_par2, new_child,
            child_after_mutation]

        cur_best_route = get_route_cost(best_par1)
        print(f"Iteration {i} Route: A -> {' -> '.join(best_par1)} -> A
            | Cost: {cur_best_route}")

    population.sort(key=get_route_cost)
    opti_route = population[0]
    opti_cost = get_route_cost(opti_route)
    print(f"\nFinal Optimal Route: A -> {' -> '.join(opti_route)} -> A"
        )
    print(f"Minimum Cost: {opti_cost}")

if __name__ == "__main__":
    genetic_algo()
```

Listing 10: TSP Genetic Algorithm Implementation

Debugging Note on Genetic Algorithms

Common Pitfalls:

- **Duplicate Parents in Next Generation:** Ensure you construct the new population array distinctly with `[parent1, parent2, ...]` rather than duplicating `parent1` twice. Loss of diversity causes premature convergence.
- **Mutation Logic:** Standard mutation should apply to the *offspring* (the crossover child) or selectively across the population, not deterministically to the elite parent, which may degrade the best known solution.

6 Constraint Satisfaction Problems (CSP)

6.1 Crossword Puzzle as a CSP

A crossword puzzle is an excellent example of a Constraint Satisfaction Problem.

- **Variables:** The blank word slots (e.g., 1 Across, 2 Down).
- **Domains:** The list of available words, categorized by the required length for each variable.
- **Constraints:** The intersecting boxes where the horizontal word and vertical word must share the exact same letter.

6.2 Solving CSP using Backtracking

Backtracking functions like a depth-first search. It assigns a word to a variable and checks if it satisfies the constraints so far. If a constraint is violated, or if the algorithm gets stuck at a future variable, it "backtracks"—erasing the assignment and trying the next available word.

6.3 Arc Consistency (AC-3) Algorithm

While Backtracking is essentially "guess and check" (which can be very slow), AC-3 acts as a **smart filter**.

[AC-3 Logic] AC-3 enforces arc consistency across all pairs of connected variables. If a variable X has a word in its domain that does not have at least one valid matching word in the domain of its neighboring variable Y (based on the intersection constraint), that word is permanently deleted from X's domain.

Why use both?

AC-3 drastically reduces the domain sizes before any guessing begins. Once AC-3 completes, Backtracking is used to finalize the solution quickly from the heavily pruned domains.

```
domains = {
    '1A': ["TELL", "TENT", "TEST", "SELF", "SAVE", "SORT"],
    '3A': ["SERVE", "SOLVE", "SENSE", "SAVED", "ASSET", "ALERT", "ARIVE
        ", "RINSE", "RIVAL", "RIVER"],
    '5A': ["EGG", "EAR", "EAT"],
    '1D': ["SERVE", "SOLVE", "SENSE", "SAVED", "ASSET", "ALERT", "ARIVE
        ", "RINSE", "RIVAL", "RIVER"],
    '2D': ["SERVE", "SOLVE", "SENSE", "SAVED", "ASSET", "ALERT", "ARIVE
        ", "RINSE", "RIVAL", "RIVER"],
    '4D': ["TELL", "TENT", "TEST", "SELF", "SAVE", "SORT"]
}

# Format: [Box1, Index1, Box2, Index2]
arcs = [
    ['1A', 0, '1D', 0], ['1D', 0, '1A', 0],
    ['1A', 2, '2D', 0], ['2D', 0, '1A', 2],
    ['1A', 3, '4D', 0], ['4D', 0, '1A', 3],
    ['3A', 1, '1D', 3], ['1D', 3, '3A', 1],
    ['3A', 3, '2D', 3], ['2D', 3, '3A', 3],
    ['3A', 4, '4D', 3], ['4D', 3, '3A', 4],
    ['5A', 0, '1D', 4], ['1D', 4, '5A', 0],
    ['5A', 2, '2D', 4], ['2D', 4, '5A', 2]
]

def revise(box_x, idx_x, box_y, idx_y):
    revised = False
    good_words = []

    for word_x in domains[box_x]:
        match_found = False
        for word_y in domains[box_y]:
            if word_x[idx_x] == word_y[idx_y]:
                match_found = True
                break
        if match_found:
            good_words.append(word_x)
        else:
            revised = True

    domains[box_x] = good_words
    return revised

def run_ac3():
    queue = list(arcs)
    while len(queue) > 0:
        box1, idx1, box2, idx2 = queue.pop(0)

        if revise(box1, idx1, box2, idx2):
            for padosi_arc in arcs:
                if padosi_arc[2] == box1 and padosi_arc[0] != box2:
```

```
                    queue.append(padosi_arc)

def solve_backtrack(assigned_words):
    if len(assigned_words) == 6:
        return assigned_words

    box_names = ['1A', '3A', '5A', '1D', '2D', '4D']
    khali_box = ""
    for box in box_names:
        if box not in assigned_words:
            khali_box = box
            break

    for word in domains[khali_box]:
        if word in assigned_words.values(): continue

        word_sahi_hai = True
        for arc in arcs:
            b1, i1, b2, i2 = arc
            if b1 == khali_box and b2 in assigned_words:
                if word[i1] != assigned_words[b2][i2]:
                    word_sahi_hai = False
                    break

        if word_sahi_hai:
            assigned_words[khali_box] = word
            result = solve_backtrack(assigned_words)
            if result: return result
            del assigned_words[khali_box] # Backtrack

    return None

if __name__ == "__main__":
    run_ac3()
    final_answer = solve_backtrack({})
    print(final_answer)
```

Listing 11: Combined AC-3 and Backtracking for Crossword CSP

6.4 Sudoku Solving using CSP Heuristics

Sudoku is a classic Constraint Satisfaction Problem where the grid cells are variables, the domain is numbers 1-9, and the constraints are the Sudoku rules (no duplicate numbers in the same row, column, or 3×3 subgrid).

[Heuristics to Speed Up CSP]

- **MRV (Minimum Remaining Values):** Select the variable (cell) that has the fewest legal values left in its domain. This minimizes the branching factor. "Fail first" principle.
- **LCV (Least Constraining Value):** Once a variable is chosen, order its available values by counting how many options they eliminate for neighboring unassigned variables. Pick the value that eliminates the *fewest* options for its neighbors, leaving maximum flexibility for future assignments.

```
import copy

def is_valid(grid, r, c, num):
```

```
    for i in range(9):
        if grid[r][i] == num or grid[i][c] == num:
            return False
    start_row, start_col = (r // 3) * 3, (c // 3) * 3
    for i in range(3):
        for j in range(3):
            if grid[start_row + i][start_col + j] == num:
                return False
    return True

def get_domain(grid, r, c):
    if grid[r][c] != 0: return []
    return [num for num in range(1, 10) if is_valid(grid, r, c, num)]

def select_mrv_cell(grid):
    min_options = 10
    best_cell = None
    best_domain = []
    for r in range(9):
        for c in range(9):
            if grid[r][c] == 0:
                domain = get_domain(grid, r, c)
                if len(domain) < min_options:
                    min_options = len(domain)
                    best_cell = (r, c)
                    best_domain = domain
    return best_cell, best_domain

def order_lcv_values(grid, r, c, domain):
    value_scores = []
    for val in domain:
        temp_grid = copy.deepcopy(grid)
        temp_grid[r][c] = val
        eliminated = 0

        for i in range(9):
            if temp_grid[r][i] == 0 and not is_valid(temp_grid, r, i,
               val): eliminated += 1
            if temp_grid[i][c] == 0 and not is_valid(temp_grid, i, c,
               val): eliminated += 1

        start_row, start_col = (r // 3) * 3, (c // 3) * 3
        for i in range(3):
            for j in range(3):
                nr, nc = start_row + i, start_col + j
                if temp_grid[nr][nc] == 0 and nr != r and nc != c:
                    if not is_valid(temp_grid, nr, nc, val):
                        eliminated += 1

        value_scores.append((val, eliminated))

    value_scores.sort(key=lambda x: x[1])
    return [item[0] for item in value_scores]

def solve_sudoku(grid):
    cell, domain = select_mrv_cell(grid)
    if cell is None: return True
    r, c = cell
```

```
    ordered_values = order_lcv_values(grid, r, c, domain)
    for val in ordered_values:
        grid[r][c] = val
        if solve_sudoku(grid): return True
        grid[r][c] = 0 # Backtrack
    return False
```

Listing 12: Sudoku Solver with MRV and LCV

7 Graph Algorithms: Shortest Paths

7.1 Dijkstra's Algorithm

Dijkstra's Algorithm finds the shortest path from a starting node to all other nodes in a weighted graph. It uses a Priority Queue (Min-Heap) to continually explore the closest known node. It acts greedily and does not work correctly with negative edge weights.

Dijkstra Time Complexity

Total Time Complexity: $O((V+E)\log V)$
Derivation:

- Initialization sets all node distances to infinity: $O(V)$.
- Extracting the minimum element from the Priority Queue happens V times. Each extraction takes $O(\log V)$. Total: $O(V \log V)$.
- Examining edges and potentially updating the Priority Queue happens E times. Each update takes $O(\log V)$. Total: $O(E \log V)$.
- Overall: $O(V) + O(V \log V) + O(E \log V) = O((V+E)\log V)$.

```
import heapq

def dijkstra(graph, v_set, source):
    dist = {vertex: float('inf') for vertex in v_set}
    parent = {vertex: None for vertex in v_set}
    dist[source] = 0

    pq = []
    heapq.heappush(pq, (0, source))

    while len(pq) > 0:
        cur_wt, u = heapq.heappop(pq)

        if cur_wt > dist[u]: continue
        if u not in graph: continue

        for neighbour, wt in graph[u]:
            if dist[u] + wt < dist[neighbour]:
                dist[neighbour] = dist[u] + wt
                parent[neighbour] = u
                heapq.heappush(pq, (dist[neighbour], neighbour))

    return dist, parent
```

```python
def res_path(parent, dest):
    path = []
    while dest is not None:
        path.append(dest)
        dest = parent[dest]
    path.reverse()
    return path
```

Listing 13: Dijkstra's Algorithm Implementation

Path Tracing Mechanism: To construct the shortest path to a destination, we start at the destination and follow the `parent` pointers backwards until we reach the `source` (which has no parent). Since the array is built backwards, we must `reverse()` it before returning.

7.2 Bellman-Ford Algorithm

Unlike Dijkstra, the Bellman-Ford algorithm does not use a Priority Queue. It uses a brute-force "chain reaction" approach: it repeatedly iterates through all edges in the graph $V - 1$ times.

Why $V - 1$ times? In the worst-case scenario (e.g., a straight-line graph), it takes $V - 1$ steps for the shortest path information to propagate from the source to the farthest node.

Time Complexity of Bellman-Ford

Total Time Complexity: $O(V \times E)$
Derivation: The outer loop runs $(V - 1)$ times. The inner loop processes all E edges. $(V - 1) \times E \approx O(V \times E)$.
Usage: Slower than Dijkstra, but **required** if the graph contains negative weight edges.

```python
# V is list of vertices, edges is list of tuples (u, v, weight)
def bellman_ford(V, edges, source):
    dist = {v: float('inf') for v in V}
    parent = {v: None for v in V}
    dist[source] = 0

    for _ in range(len(V) - 1):
        for u, v, w in edges:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                parent[v] = u

    return dist, parent
```

Listing 14: Bellman-Ford Algorithm

7.3 Floyd-Warshall Algorithm (All-Pairs Shortest Path)

While Dijkstra and Bellman-Ford are *Single-Source* algorithms (one start point), Floyd-Warshall calculates the shortest path between *every pair* of nodes simultaneously using an Adjacency Matrix.

It uses three nested loops representing `Start (i)`, `End (j)`, and `Via (k)`. Time Complexity is $O(V^3)$.

```python
# matrix[i][j] holds distance from i to j
for k in range(V):          # k = VIA (intermediate node)
    for i in range(V):      # i = START node
```

```
        for j in range(V):  # j = END node
            matrix[i][j] = min(matrix[i][j], matrix[i][k] + matrix[k][j
                ])
```

Listing 15: Floyd-Warshall Core Logic

8 Optimization & Search Problems

8.1 Brute-Force TSP using Permutations

The Traveling Salesperson Problem (TSP) can be solved perfectly using brute-force, but at a severe performance cost. By generating all possible permutations of the intermediate cities, the Time Complexity becomes $O(N!)$.

```
from itertools import permutations

def tsp_bruteforce(cities, graph, start_city):
    other_cities = [c for c in cities if c != start_city]
    min_cost = float('inf')
    best_path = []

    # Check every possible combination
    for perm in permutations(other_cities):
        path = [start_city] + list(perm) + [start_city]
        cost = sum(graph[path[i]][path[i+1]] for i in range(len(path)
            -1))

        if cost < min_cost:
            min_cost = cost
            best_path = path

    return min_cost, best_path
```

Listing 16: TSP Brute Force ($O(N!)$)

8.2 Fractional vs. 0/1 Knapsack

- **Fractional Knapsack:** You can take fractions of an item (e.g., sugar, sand). A *Greedy Approach* (sorting by Value/Weight ratio) provides the 100% optimal solution.
- **0/1 Knapsack:** You must either take the whole item or leave it entirely (e.g., a TV, a laptop). The Greedy Approach **fails** here. You must use Dynamic Programming (DP) to find the optimal solution.

```
# item format: (ratio, name, value, weight)
def custom_sort(item):
    return (-item[0], item[3]) # Sort descending by ratio. If tie,
        ascending by weight.

def fractional_knapsack(items_with_ratio, capacity):
    items_with_ratio.sort(key=custom_sort)
    total_value = 0
    cur_wt = 0
    bag = []

    for ratio, name, val, wt in items_with_ratio:
```

```
        if cur_wt + wt <= capacity:
            cur_wt += wt
            total_value += val
            bag.append(name)
        else:
            remaining_capacity = capacity - cur_wt
            total_value += (ratio * remaining_capacity)
            bag.append(name + " (Fraction)")
            break # Bag is now full

    return total_value, bag
```

Listing 17: Fractional Knapsack Greedy Logic

8.3 Algorithm Comparison Summary

1. **Greedy Best-First Search (GBFS):** Only looks at the Future heuristic $h(n)$. Prone to making bad decisions if there's an obstacle blocking the path.
2. **Dijkstra's Algorithm:** Only looks at the Past cost $g(n)$. Explores blindly in all directions.
3. **A* Algorithm:** Combines both! $f(n) = g(n) + h(n)$. Finds the most optimal path efficiently.

9 Probabilistic Reasoning & Bayesian Networks

9.1 Bayesian Networks Overview

A Bayesian Network is a probabilistic graphical model (a Directed Acyclic Graph or DAG) that represents a set of variables and their conditional dependencies. It is heavily used in AI for decision-making under uncertainty.

- **Root Nodes:** Variables with no parents (independent). Their probabilities are absolute (marginal probability).
- **Child Nodes:** Variables with parents. Their probabilities depend on the states of their parents (Conditional Probability Tables or CPTs).

Joint Probability Formula: The joint probability distribution of a Bayesian Network is the product of the conditional probabilities of all nodes given their parents.

$$P(a, m, i, e, s) = P(e) \times P(i) \times P(m \mid i, e) \times P(a \mid m) \times P(s \mid i)$$

9.2 Constructing Bayesian Networks in Python

In modern Python, `pgmpy` (Probabilistic Graphical Models in Python) is the standard library for Bayesian Networks.

Debugging Deprecated Libraries

Error: `ModuleNotFoundError: No module named 'pomegranate'`
Cause: The `pomegranate` library's API has fundamentally changed in v1.0+, breaking older code.
Error: `ImportError: 'BayesianNetwork' is deprecated. Please use 'DiscreteBayesianNetwork' instead.`
Cause: `pgmpy` renamed its core class to explicitly specify discrete distributions. Always use `DiscreteBayesianNetwork` in newer versions.

```
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD

# 1. Define Network Structure (Edges from Parent -> Child)
model = DiscreteBayesianNetwork([
    ('e', 'm'),  # Exam level -> Marks
    ('i', 'm'),  # IQ level -> Marks
    ('m', 'a'),  # Marks -> Admission
    ('i', 's')   # IQ level -> Apti score
])

# 2. Define Probabilities (CPTs)
# Exam (e) and IQ (i) - Independent
cpd_e = TabularCPD(variable='e', variable_card=2, values=[[0.7],
    [0.3]])
cpd_i = TabularCPD(variable='i', variable_card=2, values=[[0.8],
    [0.2]])

# Marks (m) - Depends on 'i' and 'e'
# Columns correspond to combinations of evidence: (i0,e0), (i0,e1), (i1
    ,e0), (i1,e1)
cpd_m = TabularCPD(variable='m', variable_card=2,
                   values=[[0.6, 0.9, 0.5, 0.8],  # m0 probabilities
                           [0.4, 0.1, 0.5, 0.2]], # m1 probabilities
                   evidence=['i', 'e'],
                   evidence_card=[2, 2])

# Apti Score (s) - Depends on 'i'
cpd_s = TabularCPD(variable='s', variable_card=2,
                   values=[[0.75, 0.4],
                           [0.25, 0.6]],
                   evidence=['i'],
                   evidence_card=[2])

# Admission (a) - Depends on 'm'
cpd_a = TabularCPD(variable='a', variable_card=2,
                   values=[[0.6, 0.9],
                           [0.4, 0.1]],
                   evidence=['m'],
                   evidence_card=[2])

# 3. Build Model
model.add_cpds(cpd_e, cpd_i, cpd_m, cpd_s, cpd_a)
if model.check_model():
    print("Model Successfully Built!")

# 4. Calculate Full Joint Probability Distribution
```

```
print("Calculating Joint Probability Distribution P(a, m, i, e, s)...")
joint_prob = cpd_e.to_factor() * cpd_i.to_factor() * cpd_m.to_factor()
    * cpd_s.to_factor() * cpd_a.to_factor()
print(joint_prob)
```

Listing 18: Bayesian Network using `pgmpy`

9.3 Pure Python Bayesian Network Implementation

Due to environment restrictions (like missing Microsoft Visual C++ 14.0 Build Tools on Windows), older libraries like `pomegranate v0.14.9` may fail to install due to "wheel build errors".

To bypass this without losing mathematical accuracy, we can implement the exact Bayesian Network logic from scratch using pure Python. This demonstrates a deep understanding of the underlying math: $P(e, i, m, s, a) = P(e) \times P(i) \times P(m|i, e) \times P(s|i) \times P(a|m)$.

```
import itertools

# 1. Root Nodes
P_e = {'e0': 0.7, 'e1': 0.3}
P_i = {'i0': 0.8, 'i1': 0.2}

# 2. Child Nodes (Conditional Probabilities)
# Format: (parent1, parent2, child): probability
P_m = {
    ('i0', 'e0', 'm0'): 0.6, ('i0', 'e0', 'm1'): 0.4,
    ('i0', 'e1', 'm0'): 0.9, ('i0', 'e1', 'm1'): 0.1,
    ('i1', 'e0', 'm0'): 0.5, ('i1', 'e0', 'm1'): 0.5,
    ('i1', 'e1', 'm0'): 0.8, ('i1', 'e1', 'm1'): 0.2
}

P_s = {
    ('i0', 's0'): 0.75, ('i0', 's1'): 0.25,
    ('i1', 's0'): 0.40, ('i1', 's1'): 0.60
}

P_a = {
    ('m0', 'a0'): 0.6, ('m0', 'a1'): 0.4,
    ('m1', 'a0'): 0.9, ('m1', 'a1'): 0.1
}

# 3. Calculate Joint Probability
print("Calculating P(a, m, i, e, s):")
variable_states = [['a0', 'a1'], ['m0', 'm1'], ['i0', 'i1'], ['e0', 'e1
    '], ['s0', 's1']]
combinations = list(itertools.product(*variable_states))

for combo in combinations:
    a, m, i, e, s = combo
    prob_e = P_e[e]
    prob_i = P_i[i]
    prob_m = P_m[(i, e, m)]
    prob_s = P_s[(i, s)]
    prob_a = P_a[(m, a)]

    # Chain Rule
    joint_prob = prob_a * prob_m * prob_i * prob_e * prob_s
    print(f"P({a}, {m}, {i}, {e}, {s}) = {joint_prob:.5f}")
```

Listing 19: Pure Python Bayesian Network (No Libraries)

This code manually iterates through all 32 possible states (2^5), applies the Bayesian Chain Rule via dictionary lookups, and produces the exact same output as the dedicated libraries without any compilation overhead.

10 Uncertainty & Fuzzy Logic

10.1 Logical Deduction vs Induction

- **Deduction (Formal Logic):** Deriving exact conclusions from known premises. The result is 100% certain.
- **Induction (Informal Logic):** Based on probability and estimation. Conclusions are "probable" or "plausible" but not 100% fixed.

10.2 Why Probability over First-Order Logic (FOL)?

Classical FOL struggles with uncertainty. For example, the rule "Toothache implies Cavity" is flawed because a toothache can be caused by gum disease or other issues. Creating an exceptionless rule in FOL requires listing every possible cause, which is impractical. Probability allows us to handle theoretical and practical ignorance, enabling both **Prediction** (Cause $\rightarrow$ Effect) and **Diagnosis** (Effect $\rightarrow$ Cause).

10.3 Bayesian Inferences

Using the classic **Burglar Alarm Example** (An alarm can be triggered by a Burglary or an Earthquake. Neighbors John and Mary may or may not call when the alarm rings):

1. **Diagnostic (Effect to Cause):** Given John calls, what is the probability of a Burglary? ($P(\text{Burglary} \mid \text{JohnCalls})$).
2. **Causal (Cause to Effect):** Given a Burglary, what is the probability John calls? ($P(\text{JohnCalls} \mid \text{Burglary})$).
3. **Inter-causal ("Explaining Away"):** If the alarm rings, the probability of Burglary goes up. But if we discover an Earthquake also happened, the probability of Burglary drops significantly because the Earthquake "explains away" the alarm.
4. **Mixed:** Combining both causal and diagnostic inferences.

10.4 Fuzzy Logic

Fuzzy logic handles concepts that are "vague" or not strictly True/False (e.g., Is this person "fat"? Is the diet "high"?). Truth values exist on a spectrum between 0 and 1.

[Fuzzy Operations]

- **AND:** Takes the Minimum value. $\min(A, B)$.
- **OR:** Takes the Maximum value. $\max(A, B)$.

Example (Cardiac Risk): If Diet-Low = 0.4 and Exercise-High = 0.5. Rule: `If (Diet is Low) AND (Exercise is High) THEN Risk is Low`. Value: $\min(0.4, 0.5) = 0.4$. (Low Risk is 0.4).

If Diet-High = 0.6 and Exercise-Low = 0.5. Rule: `If (Diet is High) OR (Exercise is Low) THEN Risk is High`. Value: $\max(0.6, 0.5) = 0.6$. (High Risk is 0.6).

These fuzzy values are then combined via **Defuzzification** (using calculus/integration over membership functions) to produce a crisp percentage (e.g., 47.5% chance of Heart Disease).

11 Advanced CSP: MRV and LCV

When solving Constraint Satisfaction Problems (like Sudoku or Map Coloring), AI uses heuristics to decide what to do next to avoid getting stuck.

11.1 MRV (Minimum Remaining Values)

The "Fail-First" Heuristic.

- **Rule:** Always select the variable (cell) that has the *fewest* valid options left in its domain.
- **Why?** If a cell only has 2 possible numbers and you leave it for last, the board might fill up in a way that blocks both numbers, creating a dead end. Pick the most constrained variable first.

11.2 LCV (Least Constraining Value)

The "Live and Let Live" Heuristic.

- **Rule:** Once a variable is chosen, order its available values by how many options they eliminate for neighboring variables. Pick the value that eliminates the *fewest* options for its neighbors.
- **Why?** This leaves maximum flexibility for future assignments, reducing the need to backtrack.

12 Local Search Algorithms

Local search algorithms are used when the path to the goal doesn't matter; only the final goal state matters. They operate using a single current node and move to neighbors.

12.1 Hill Climbing Algorithm (Steepest Ascent)

Hill climbing is a greedy local search. It looks at all immediate neighbors and moves to the one that improves the current state the most.

The Trap (Local Maxima/Minima): If all neighbors are worse or equal to the current state, the algorithm gets stuck, believing it has reached the peak (even if a higher peak exists elsewhere).

```
# Assuming move_gen() generates valid neighbors
# and get_heuristic() calculates misplaced tiles

def hill_climb(START, GOAL):
    cur_best_board = START
    cur_best_val = get_heuristic(START)

    while True:
        if cur_best_val == 0:
            print("Goal Reached!")
            break
```

```
        row_of_zero, col_of_zero = get_zero_location(cur_best_board)
        neighbors = move_gen(row_of_zero, col_of_zero, cur_best_board)

        # Find steepest ascent (lowest error) among neighbors
        best_neighbor = neighbors[0]
        best_neighbor_val = get_heuristic(best_neighbor)

        for neighbor in neighbors:
            h_val = get_heuristic(neighbor)
            if h_val < best_neighbor_val:
                best_neighbor_val = h_val
                best_neighbor = neighbor

        # The Trap Check
        if best_neighbor_val >= cur_best_val:
            print("Stuck at Local Minima. Goal not reachable.")
            break

        # Move
        cur_best_board = best_neighbor
        cur_best_val = best_neighbor_val
```

Listing 20: 8-Puzzle Hill Climbing Engine

12.2 Iterated (Random-Restart) Hill Climbing

To overcome the Local Maxima trap, we use Random-Restart Hill Climbing. If the algorithm gets stuck, we randomly shuffle the state and try again.

Generating Solvable 8-Puzzle States

Critical Concept: A completely random shuffle of an 8-puzzle has a 50% chance of being unsolvable mathematically. To generate a guaranteed solvable random state, start from the GOAL state and simulate N random valid moves backwards.

```
import random

def get_random_board(GOAL):
    # Start from goal and make 40 random moves
    board = [row[:] for row in GOAL]
    for _ in range(40):
        r, c = get_zero_location(board)
        neighbors = move_gen(r, c, board)
        board = random.choice(neighbors)
    return board

def iterative_hill_climb(GOAL, max_iterations=15):
    for i in range(1, max_iterations + 1):
        START = get_random_board(GOAL)
        cur_best_board = START
        cur_best_val = get_heuristic(START)

        while True:
            if cur_best_val == 0:
                print(f"Goal reached in iteration {i}!")
                return # Exit completely
```

```
            # ... (Hill climbing neighbor generation and evaluation
                here) ...

            if best_neighbor_val >= cur_best_val:
                print(f"Stuck at local maxima. Restarting...")
                break # Break inner loop, move to next iteration

            cur_best_board = best_neighbor
            cur_best_val = best_neighbor_val
```

Listing 21: Iterated Hill Climbing with Guaranteed Solvable Restarts

13 Crossword Puzzle as a CSP

A Crossword puzzle is a classic Constraint Satisfaction Problem (CSP). The variables are the word slots (e.g., 1A, 3A, 1D). The domains are lists of possible words. The constraints are the intersections (arcs) where a letter from one word must match a letter from an intersecting word.

We solve this using two approaches: Backtracking and AC-3.

13.1 Backtracking (Smart Hit & Trial)

Backtracking places a word, checks if it violates any intersection rules, and moves forward. If it hits a dead end (no valid words for the next slot), it backtracks and tries the next word in the previous slot.

13.2 AC-3 (Arc Consistency 3)

AC-3 is a preprocessing step. It filters the domains *before* assigning words.

1. Put all arcs (intersection rules) in a Queue.
2. Pop an arc (e.g., $1A \to 1D$).
3. Check every word in $1A$'s domain. If a word (e.g., "SORT") starts with 'S', but there are no words in $1D$'s domain starting with 'S', delete "SORT" from $1A$'s domain.
4. If $1A$'s domain changes, put all arcs pointing to $1A$ back into the queue.
5. Repeat until the queue is empty.

This drastically reduces the search space, often leaving only the correct words in the domains.

14 Debugging Notes: 8-Puzzle Recursive Implementation

A recursive version of Hill Climbing was implemented, but it contained several common Python translation bugs from C++/Java.

Common Implementation Bugs

- **Array Length:** Python uses `len(grid)` instead of `grid.size()`.
- **Return Statements in Recursion:** When a recursive function calls itself, it must `return` the result of that call (e.g., `return solve(best_neighbor)`). Otherwise, the final `True/False` result is lost as it bubbles back up the stack.
- **Empty Tile Logic:** The heuristic should not count the empty tile `0` as a misplaced tile.

15 Adversarial Search & Game Playing

15.1 Minimax Algorithm

Minimax is used for two-player adversarial games (like Tic-Tac-Toe or Chess). The algorithm explores the game tree to its leaves (or a specific depth) to determine the best possible move, assuming the opponent plays optimally.

- **Maximizer (AI):** Tries to maximize the score (e.g., +1 for win).
- **Minimizer (Human):** Tries to minimize the score (e.g., -1 for win).

The Depth Trick (Lazy vs. Smart AI)

Instead of returning a simple +1 for a win and -1 for a loss, advanced Minimax implementations return `10 - depth` for a win and `depth - 10` for a loss.
Why? Without depth factoring, an AI considers a win in 1 move to be exactly as good as a win in 5 moves. By subtracting depth, the AI is motivated to win as quickly as possible (higher score) and lose as slowly as possible (stall the game).

15.2 Alpha-Beta Pruning

Minimax explores *every* possible branch, which is extremely slow (e.g., Tic-Tac-Toe has $9! = 362,880$ possibilities). Alpha-Beta Pruning cuts off branches that are guaranteed to be worse than the current best option.

- **Alpha (α):** The best (highest) score the Maximizer can guarantee so far. Starts at $-\infty$.
- **Beta (β):** The best (lowest) score the Minimizer can guarantee so far. Starts at $+\infty$.

The Cut-off Rule: `if beta <= alpha: break`
Analogy: If the Maximizer already found a path guaranteeing +5 points ($\alpha = 5$), and starts exploring a new path where the Minimizer can force the score down to +3 ($\beta = 3$), the Maximizer will immediately stop exploring that path because $3 \leq 5$. It's a waste of time to explore a path the opponent will make worse than an already known alternative.

```
def min_max(board, depth, alpha, beta):
    winner = check_win(board)
    if winner == 'O': return 10 - depth
    if winner == 'X': return depth - 10
    if is_board_full(board): return 0

    # Maximizer's Turn (AI 'O')
    if depth % 2 == 0:
```

```
        best_score = -1000
        for i in range(3):
            for j in range(3):
                if board[i][j] == ' ':
                    board[i][j] = 'O'
                    score = min_max(board, depth + 1, alpha, beta)
                    board[i][j] = ' ' # Backtrack

                    best_score = max(best_score, score)
                    alpha = max(alpha, best_score)
                    if beta <= alpha:
                        break # Prune branch
        return best_score

    # Minimizer's Turn (User 'X')
    else:
        best_score = 1000
        # ... Similar logic but finding min(), updating beta, and
            breaking if beta <= alpha ...
        return best_score
```

Listing 22: Minimax with Alpha-Beta Pruning

Note on Game Flow: When the AI calculates its move, it places its mark and calls `min_max(board, depth=1, ...)`. Depth 1 is passed because the next simulated turn in the recursive tree belongs to the Minimizer (odd depth).

16 The 8-Queens Problem

The goal of the 8-Queens problem is to place 8 queens on an 8x8 chessboard such that no two queens threaten each other. They cannot share the same row, column, or diagonal.

We can solve this efficiently using **Random-Restart Hill Climbing**.

16.1 Smart Board Representation

Instead of a full 8×8 matrix, we use a 1D Array (List) of size 8.

- **Index:** Represents the Column (0 to 7).
- **Value:** Represents the Row where the queen is placed (0 to 7).

Example: `board = [4, 2, 0, 6, 1, 7, 5, 3]` means Queen in Col 0 is at Row 4, Col 1 is at Row 2, etc.
Massive Advantage: By design, every queen is in a different column. We completely eliminate the possibility of column collisions and never have to check for them!

16.2 The Cost Function (Referee)

Our goal is to minimize clashes (Cost = 0). Since column clashes are impossible, we only check for:

1. **Row Clash:** `board[i] == board[j]`
2. **Diagonal Clash:** The absolute difference of rows equals the absolute difference of columns. `abs(board[i] - board[j]) == abs(i - j)`

16.3 Algorithm Logic

1. **Random Restart (Outer Loop):** Generate a random board. If the climbing phase gets stuck, reset to a new random board.
2. **Hill Climbing (Inner Loop):** Pick a queen (column) and test moving it to every other row (0-7) in that column. Track the move that results in the lowest number of total clashes across the board.
3. Commit to the best move. If no move improves the score, break (we are stuck in a Local Minima), triggering a random restart.

```
import random

def calc_cost(board):
    cost = 0
    n = len(board)
    for i in range(n):
        for j in range(i + 1, n):
            # Check row collision OR diagonal collision
            if board[i] == board[j] or abs(board[i] - board[j]) == abs(
                i - j):
                cost += 1
    return cost

def solve(n):
    while True: # Outer Loop (Random Restarts)
        board = [random.randint(0, n - 1) for _ in range(n)]

        while True: # Inner Loop (Hill Climbing)
            curr_cost = calc_cost(board)
            if curr_cost == 0:
                return board # Solved!

            best_board = list(board) # Must use list() to copy, not
                reference
            min_cost = curr_cost

            for col in range(n):
                orig_row = board[col]
                for row in range(n): # Try every row for this queen
                    if row != orig_row:
                        board[col] = row
                        new_cost = calc_cost(board)

                        if new_cost < min_cost:
                            min_cost = new_cost
                            best_board = list(board) # Save the best
                                move found

                board[col] = orig_row # Backtrack to original position

            # If stuck in Local Minima
            if min_cost >= curr_cost:
                break

            # Move up the hill
            board = best_board
```

Listing 23: 8-Queens using Random-Restart Hill Climbing

17 Graph Algorithms & Optimization

17.1 A* Algorithm (Pathfinding)

A* is a smart search algorithm that finds the shortest path by combining actual cost and an estimated cost (heuristic). **Formula:** $f(n) = g(n) + h(n)$

- $g(n)$: Actual cost from the start node to the current node.
- $h(n)$: Estimated cost from the current node to the goal (Heuristic).
- $f(n)$: Total estimated cost. A* always explores the path with the lowest $f(n)$.

17.2 Bellman-Ford Algorithm

Used for finding the shortest path from a single source node to all other nodes in a weighted graph.

- **Advantage over Dijkstra:** Can handle negative weight edges.
- **Logic (Edge Relaxation):** For $V - 1$ iterations (where V is number of vertices), check every edge (u, v) with weight w. If `dist[u] + w < dist[v]`, update `dist[v] = dist[u] + w`.

18 Genetic Algorithms (Traveling Salesperson Problem)

Genetic algorithms solve optimization problems by mimicking biological evolution (Survival of the Fittest).

1. **Population:** Generate random possible solutions (routes).
2. **Fitness:** Evaluate each solution. For TSP, lower cost = higher fitness.
3. **Crossover (Breeding):** Combine the two best solutions to create a new "child" solution. (e.g., Take the first half of the route from Parent 1, and fill the rest in order from Parent 2).
4. **Mutation:** Introduce random changes (e.g., swap two cities in the route) to maintain genetic diversity and avoid local optima.

19 Advanced CSP Formulation: Timetable Scheduling

Scheduling problems (e.g., assigning Professors to Classes at specific Times) are complex CSPs.

- **Variables:** The entities needing assignment (e.g., Classes).
- **Domains:** Tuples of possible assignments (e.g., `(Professor, Time)` pairs).
- **Node Consistency (Pre-filtering):** Remove domain values that are impossible (e.g., a Professor who doesn't teach that subject, or is unavailable at that time).

- **Constraints (Arcs):** Rules governing intersections. E.g., If `Prof(Class1) == Prof(Class2)`, then `Time(Class1) != Time(Class2)`.

Backtracking places a word, checks if it violates any intersection rules, and moves forward. If it hits a dead end (no valid words for the next slot), it backtracks and tries the next word in the previous slot.

19.1 AC-3 (Arc Consistency 3)

AC-3 is a preprocessing step. It filters the domains *before* assigning words.

1. Put all arcs (intersection rules) in a Queue.
2. Pop an arc (e.g., $1A \to 1D$).
3. Check every word in $1A$'s domain. If a word (e.g., "SORT") starts with 'S', but there are no words in $1D$'s domain starting with 'S', delete "SORT" from $1A$'s domain.
4. If $1A$'s domain changes, put all arcs pointing to $1A$ back into the queue.
5. Repeat until the queue is empty.

This drastically reduces the search space, often leaving only the correct words in the domains.

20 Debugging Notes: 8-Puzzle Recursive Implementation

A recursive version of Hill Climbing was implemented, but it contained several common Python translation bugs from C++/Java.

Common Implementation Bugs

- **Array Length:** Python uses `len(grid)` instead of `grid.size()`.
- **Return Statements in Recursion:** When a recursive function calls itself, it must `return` the result of that call (e.g., `return solve(best_neighbor)`). Otherwise, the final `True/False` result is lost as it bubbles back up the stack.
- **Empty Tile Logic:** The heuristic should not count the empty tile `0` as a misplaced tile.

21 Adversarial Search & Game Playing

21.1 Minimax Algorithm

Minimax is used for two-player adversarial games (like Tic-Tac-Toe or Chess). The algorithm explores the game tree to its leaves (or a specific depth) to determine the best possible move, assuming the opponent plays optimally.

- **Maximizer (AI):** Tries to maximize the score (e.g., +1 for win).
- **Minimizer (Human):** Tries to minimize the score (e.g., -1 for win).

The Depth Trick (Lazy vs. Smart AI)

Instead of returning a simple +1 for a win and -1 for a loss, advanced Minimax implementations return `10 - depth` for a win and `depth - 10` for a loss.
Why? Without depth factoring, an AI considers a win in 1 move to be exactly as good as a win in 5 moves. By subtracting depth, the AI is motivated to win as quickly as possible (higher score) and lose as slowly as possible (stall the game).

21.2 Alpha-Beta Pruning

Minimax explores *every* possible branch, which is extremely slow (e.g., Tic-Tac-Toe has $9! = 362,880$ possibilities). Alpha-Beta Pruning cuts off branches that are guaranteed to be worse than the current best option.

- **Alpha (α):** The best (highest) score the Maximizer can guarantee so far. Starts at $-\infty$.
- **Beta (β):** The best (lowest) score the Minimizer can guarantee so far. Starts at $+\infty$.

The Cut-off Rule: `if beta <= alpha: break`
Analogy: If the Maximizer already found a path guaranteeing +5 points ($\alpha = 5$), and starts exploring a new path where the Minimizer can force the score down to +3 ($\beta = 3$), the Maximizer will immediately stop exploring that path because $3 \leq 5$. It's a waste of time to explore a path the opponent will make worse than an already known alternative.

```
def min_max(board, depth, alpha, beta):
    winner = check_win(board)
    if winner == 'O': return 10 - depth
    if winner == 'X': return depth - 10
    if is_board_full(board): return 0

    # Maximizer's Turn (AI 'O')
    if depth % 2 == 0:
        best_score = -1000
        for i in range(3):
            for j in range(3):
                if board[i][j] == ' ':
                    board[i][j] = 'O'
                    score = min_max(board, depth + 1, alpha, beta)
                    board[i][j] = ' ' # Backtrack

                    best_score = max(best_score, score)
                    alpha = max(alpha, best_score)
                    if beta <= alpha:
                        break # Prune branch
        return best_score

    # Minimizer's Turn (User 'X')
    else:
        best_score = 1000
        # ... Similar logic but finding min(), updating beta, and
            breaking if beta <= alpha ...
        return best_score
```

Listing 24: Minimax with Alpha-Beta Pruning

Note on Game Flow: When the AI calculates its move, it places its mark and calls `min_max(board, depth=1, ...)`. Depth 1 is passed because the next simulated turn in the recursive tree belongs to the Minimizer (odd depth).

22 The 8-Queens Problem

The goal of the 8-Queens problem is to place 8 queens on an 8x8 chessboard such that no two queens threaten each other. They cannot share the same row, column, or diagonal.

We can solve this efficiently using **Random-Restart Hill Climbing**.

22.1 Smart Board Representation

Instead of a full 8×8 matrix, we use a 1D Array (List) of size 8.

- **Index:** Represents the Column (0 to 7).
- **Value:** Represents the Row where the queen is placed (0 to 7).

Example: `board = [4, 2, 0, 6, 1, 7, 5, 3]` means Queen in Col 0 is at Row 4, Col 1 is at Row 2, etc.
Massive Advantage: By design, every queen is in a different column. We completely eliminate the possibility of column collisions and never have to check for them!

22.2 The Cost Function (Referee)

Our goal is to minimize clashes (Cost = 0). Since column clashes are impossible, we only check for:

1. **Row Clash:** `board[i] == board[j]`
2. **Diagonal Clash:** The absolute difference of rows equals the absolute difference of columns. `abs(board[i] - board[j]) == abs(i - j)`

22.3 Algorithm Logic

1. **Random Restart (Outer Loop):** Generate a random board. If the climbing phase gets stuck, reset to a new random board.
2. **Hill Climbing (Inner Loop):** Pick a queen (column) and test moving it to every other row (0-7) in that column. Track the move that results in the lowest number of total clashes across the board.
3. Commit to the best move. If no move improves the score, break (we are stuck in a Local Minima), triggering a random restart.

```
import random

def calc_cost(board):
    cost = 0
    n = len(board)
    for i in range(n):
        for j in range(i + 1, n):
            # Check row collision OR diagonal collision
            if board[i] == board[j] or abs(board[i] - board[j]) == abs(
                i - j):
                 cost += 1
    return cost

def solve(n):
    while True: # Outer Loop (Random Restarts)
        board = [random.randint(0, n - 1) for _ in range(n)]
```

```
        while True: # Inner Loop (Hill Climbing)
            curr_cost = calc_cost(board)
            if curr_cost == 0:
                return board # Solved!

            best_board = list(board) # Must use list() to copy, not
               reference
            min_cost = curr_cost

            for col in range(n):
                orig_row = board[col]
                for row in range(n): # Try every row for this queen
                    if row != orig_row:
                        board[col] = row
                        new_cost = calc_cost(board)

                        if new_cost < min_cost:
                            min_cost = new_cost
                            best_board = list(board) # Save the best
                               move found

                board[col] = orig_row # Backtrack to original position

            # If stuck in Local Minima
            if min_cost >= curr_cost:
                break

            # Move up the hill
            board = best_board
```

Listing 25: 8-Queens using Random-Restart Hill Climbing

23 Graph Algorithms & Optimization

23.1 A* Algorithm (Pathfinding)

A* is a smart search algorithm that finds the shortest path by combining actual cost and an estimated cost (heuristic). **Formula:** $f(n) = g(n) + h(n)$

- $g(n)$: Actual cost from the start node to the current node.
- $h(n)$: Estimated cost from the current node to the goal (Heuristic).
- $f(n)$: Total estimated cost. A* always explores the path with the lowest $f(n)$.

23.2 Bellman-Ford Algorithm

Used for finding the shortest path from a single source node to all other nodes in a weighted graph.

- **Advantage over Dijkstra:** Can handle negative weight edges.
- **Logic (Edge Relaxation):** For $V - 1$ iterations (where V is number of vertices), check every edge (u, v) with weight w. If `dist[u] + w < dist[v]`, update `dist[v] = dist[u] + w`.

24 Genetic Algorithms (Traveling Salesperson Problem)

Genetic algorithms solve optimization problems by mimicking biological evolution (Survival of the Fittest).

1. **Population:** Generate random possible solutions (routes).
2. **Fitness:** Evaluate each solution. For TSP, lower cost = higher fitness.
3. **Crossover (Breeding):** Combine the two best solutions to create a new "child" solution. (e.g., Take the first half of the route from Parent 1, and fill the rest in order from Parent 2).
4. **Mutation:** Introduce random changes (e.g., swap two cities in the route) to maintain genetic diversity and avoid local optima.

25 Advanced CSP Formulation: Timetable Scheduling

Scheduling problems (e.g., assigning Professors to Classes at specific Times) are complex CSPs.

- **Variables:** The entities needing assignment (e.g., Classes).
- **Domains:** Tuples of possible assignments (e.g., `(Professor, Time)` pairs).
- **Node Consistency (Pre-filtering):** Remove domain values that are impossible (e.g., a Professor who doesn't teach that subject, or is unavailable at that time).
- **Constraints (Arcs):** Rules governing intersections. E.g., If `Prof(Class1) == Prof(Class2)`, then `Time(Class1) != Time(Class2)`.

Applying AC-3 (Arc Consistency) to this formulation filters out conflicting schedules before backtracking search even begins.